claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-a0d69da7de6899717.jsonl`
- SHA-256: `1c06f815f4bc6cb9a82723f8c663fd3858c60ba8b781929bf0dceb1076be87bd`
- Source modified: `2026-06-16T06:35:52+00:00`
- Imported at: `2026-07-05T16:48:29+00:00`
- project: `wf_ef85c152-821`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

Transcript

1. user (2026-06-16T06:32:58.969Z)

You are a SKEPTICAL reviewer. Confirm or REFUTE this audit finding against the ACTUAL code in the worktree C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (cite file:line). Decide isMustFix = is this a REAL issue that must be fixed before relying on the suite, vs a non-issue / nice-to-have / already-handled? Default to 'refuted' or 'partial' (isMustFix=false) unless you can confirm it in the code.

FINDING [DETERMINISM / cross-OS (Golden Regression Fixtures, PRs #186/#189/#191/#192, worktree gf-doc @ 8d3e185) / high]: Time-origin reset is NOT metric-invariant for non-whole-second t0 ? contradicts the 'provably metric-invariant / bit-identical scores' claim, and the worst delta lands past the 1e-6 cliff
EVIDENCE: docs/GOLDEN_REGRESSION_FIXTURES.md:94 ('Time-origin reset ... Provably metric-invariant ... bit-identical scores'); backend/eval/golden_fixtures.py:619-623 + :583-586 assert the same. But the reset subtracts t0_sec in float and writes labels/trajectory at 6dp (_fmt, golden_fixtures.py:195-207,274-279), then the STRICT gt_loader re-parses the rounded text ? so when t0_sec is non-terminating, the reset vs no-reset scores diverge. I swept frame offsets 0..39 (fps=30) through freeze_real_fixture(time_origin_reset=True vs False): score equality BROKE on 26/40 offsets; e.g. offset=7 gives mean_start_error 0.033333 (reset) vs 0.033334 (no reset). Worst measured |delta|=1.000000000001e-06, i.e. STRICTLY greater than _FLOAT_TOL=1e-6 (golden_fixtures.py:91) used by compare_expected.approx (golden_fixtures.py:786-789) ? so this can flip a parse-compare digit/result. The committed test only passes because it hard-codes offset_frames=9000 ? 9000/30=300.0s exactly (tests/test_golden_real_fixtures.py:175), a whole-second offset that never hits the cliff.
PROPOSED FIX: Either (a) weaken the doc/code claim to 'metric-invariant up to the 6dp serialization rounding' and prove the bound is < _FLOAT_TOL, or (b) make it actually invariant: compute t0 so the shift is an exact 6dp / exact-frame quantity (e.g. reset by an integer number of frames only and derive t0_sec consistently, or round t0_sec to 6dp before subtracting from both streams). Add a parametrized test over fractional-second offsets (e.g. 7,11,13 frames) asserting score equality AND |delta| <= _FLOAT_TOL, replacing/augmenting the single 9000-frame case.

2. assistant (2026-06-16T06:33:01.264Z)

I'll verify this finding against the actual code. Let me start by reading the relevant files.

3. user (2026-06-16T06:33:03.431Z)

854db56 docs(eval): golden-fixtures discoverability ? CODE_MAP §0/§2.3 + GOLDEN_DATA_SHARING cross-link (DOC)
feat/golden-fixtures-doc

4. user (2026-06-16T06:33:04.096Z)

1	"""Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2

```
3       `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4       replay code path** used by both the freeze tool and the characterization gate, and it
5       deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6       variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7       across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8       only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9       merges overlaps, and uses **no numpy**.
10
11      The replay reuses the already-shipped eval stack VERBATIM::
12
13          parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14              ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15              ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17      Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
18
19      * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20        provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21      * Every fixture is tagged ``kind="synthetic"``, ``minors_free=true``,
``license="synthetic"``.
22      * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23        person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24      * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25        provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26        ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28      §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29      suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30      frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31      that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32      field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34      Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-
only
35      for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36      """
37      from __future__ import annotations
38
39      import argparse
40      import json
41      import math
42      import os
43      import re
44      import sys
45      from dataclasses import dataclass
46      from pathlib import Path
47      from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-------------
50      from backend.eval import distill_local
51      from backend.eval.experiment import CONTROL, VideoCandidates, predict
52      from backend.eval.gt_loader import load_gt_intervals
53      from backend.eval.metrics import score
```

```
    from backend.eval.regression import gt_hash
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    Interval = Tuple[float, float]

    #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
    #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
    CURRENT_SCHEMA = 1

    #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
    #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
    FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"

    #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
    #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
    #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
    #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
    #: re-states these in its provenance; the replay reads them from there.
    PINNED_VT = 0.02
    PINNED_MERGE_GAP = 2.0
    PINNED_MIN_WINDOW = 2.0
    PINNED_MAX_WINDOW = 0.0  # 0 = bounded-windowing OFF (W1), matching the SERVED default

    #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
    #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
    SHUTTLE_FEATURES: Tuple[str, ...] = (
        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
    )
    assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
        "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
    )

    #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
    #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

    _FLOAT_ROUND_DP = 6  # all written floats rounded to this many decimals (determinism)
    _FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats

    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")


    # ============================================================================
    # Synthetic trajectory generation (deterministic ? NO random)
    # ============================================================================


    @dataclass(frozen=True)
    class RallySpec:
        """One in-flight rally segment: the shuttle oscillates across the net midline.

        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
```

```
        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
        """

        start_frame: int
        n_frames: int
        period: float = 30.0
        amplitude: float = 400.0


    @dataclass(frozen=True)
    class GapSpec:
        """Dead-time between rallies ? what separates one window from the next.

        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
        end an active run so the next rally forms a distinct window.
        """

        start_frame: int
        n_frames: int
        kind: str = "still"  # "still" | "occluded"


    @dataclass(frozen=True)
    class FixtureSpec:
        """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

        ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
        "labels.csv". For a synthetic fixture they line up with the rallies by construction,
        which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
        field-quality one (§6).
        """

        slug: str
        description: str
        fps: float
        frame_width: float
        frame_height: float
        segments: Tuple[Any, ...]  # ordered (RallySpec | GapSpec)
        gts: Tuple[Interval, ...]
        net_midline: float = 640.0


    def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
                      ) -> List[Tuple[int, int, float, float]]:
        """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
            # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
            y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
            rows.append((spec.start_frame + i, 1, x, y))
        return rows


    def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
                    ) -> List[Tuple[int, int, float, float]]:
        """Dead-time rows: resting-visible (still) or absent (occluded)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
```

```
        for i in range(spec.n_frames):
            if spec.kind == "occluded":
                rows.append((spec.start_frame + i, 0, 0.0, 0.0))
            elif spec.kind == "still":
                rows.append((spec.start_frame + i, 1, net_midline, y_mid))
            else:  # pragma: no cover - guarded by builder
                raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
        return rows


    def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
        """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
        rows: List[Tuple[int, int, float, float]] = []
        for seg in spec.segments:
            if isinstance(seg, RallySpec):
                rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
            elif isinstance(seg, GapSpec):
                rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
            else:  # pragma: no cover - guarded by dataclass shape
                raise TypeError(f"unknown segment type {type(seg).__name__}")
        rows.sort(key=lambda r: r[0])
        return rows


    def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
        """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
        lines = ["Frame,Visibility,X,Y"]
        for frame, vis, x, y in rows:
            lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
        """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
        lines = ["start,end"]
        for s, e in gts:
            lines.append(f"{_fmt(s)},{_fmt(e)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    # ============================================================================
    # Built-in synthetic scenarios (the 3 committed slugs)
    # ============================================================================


    def _fr(seconds: float, fps: float) -> int:
        return int(round(seconds * fps))


    def builtin_specs() -> List[FixtureSpec]:
        """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
        windowing behaviour (one window / dead-time split / occlusion split)."""
        fps = 30.0
        fw, fh = 1280.0, 720.0
        net = 640.0

        # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
        single = FixtureSpec(
            slug="clean_single_rally",
```

```
            description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(RallySpec(start_frame=0, n_frames=120),),
            # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
            gts=((0.0, 4.0),),
        )

        # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
        #    low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
        r0 = RallySpec(start_frame=0, n_frames=120)
        gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
        r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
        multi = FixtureSpec(
            slug="multi_rally_with_deadtime",
            description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(r0, gap_still, r1),
            gts=((0.0, 4.0), (7.0, 11.0)),
        )

        # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
        #    visibility break ? proves occlusion (not just low velocity) separates windows.
        o0 = RallySpec(start_frame=0, n_frames=120)
        gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
        o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
        occ = FixtureSpec(
            slug="occlusion_break",
            description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(o0, gap_occ, o1),
            gts=((0.0, 4.0), (7.0, 11.0)),
        )

        return [single, multi, occ]


    def _spec_by_slug() -> Dict[str, FixtureSpec]:
        return {s.slug: s for s in builtin_specs()}


    # ============================================================================
    # The shared replay (CONTROL / pure-stdlib path)
    # ============================================================================


    def _fmt(x: float) -> str:
        """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        if r == 0.0:
            r = 0.0
        return f"{r:.6f}"


    def _round(x: float) -> float:
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        return 0.0 if r == 0.0 else r


    def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
```

```
        """The windowing proposal tuple, read from the fixture provenance when present (the
pinned
        params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
        if prov and "windowing" in prov:
            w = prov["windowing"]
            return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
        return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)


    def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
        """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.

        Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
        ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

            parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
            cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).

        Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
        control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
        merges overlaps; NO numpy is touched.
        """
        fdir = _resolve_dir(slug_or_dir)
        slug = fdir.name
        traj_csv = fdir / "trajectory.csv"
        labels_csv = fdir / "labels.csv"
        prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None

        fps, fw = _fps_fw(prov, slug)
        iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
        proposal = _pinned_proposal(prov)

        intervals, feature_rows = distill_local.build_candidates(
            str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)

        # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
        # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
        # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
        features: Dict[str, List[float]] = {
            name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
        }
        gts = load_gt_intervals(str(labels_csv), strict=True)
        vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)

        preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
        sc = score(preds, gts, iou)

        candidates: List[Dict[str, Any]] = []
        for j, (s, e) in enumerate(intervals):
            shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
            # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
            shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
```

```
            candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})

        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "iou": _round(iou),
            "candidates": candidates,
            "control_segments": [[_round(s), _round(e)] for s, e in preds],
            "score": {
                "iou_threshold": _round(sc.iou_threshold),
                "num_pred": int(sc.num_pred),
                "num_gt": int(sc.num_gt),
                "true_positives": int(sc.true_positives),
                "false_positives": int(sc.false_positives),
                "false_negatives": int(sc.false_negatives),
                "precision": _round(sc.precision),
                "recall": _round(sc.recall),
                "f1": _round(sc.f1),
                "mean_iou": _round(sc.mean_iou),
                "mean_start_error": _round(sc.mean_start_error),
                "mean_end_error": _round(sc.mean_end_error),
            },
            "gt_hash": gt_hash(gts),
        }


    # ============================================================================
    # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
    # ============================================================================


    def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
                    ) -> Dict[str, Any]:
        """The committed provenance block ? pins the windowing + privacy/licensing tags
(§4c, §5)."""
        desc = ""
        if isinstance(spec_or_dir, FixtureSpec):
            desc = spec_or_dir.description
        return {
            "slug": slug,
            "schema_version": CURRENT_SCHEMA,
            "kind": "synthetic",
            "license": "synthetic",
            "minors_free": True,
            "description": desc,
            "fps": _round(fps),
            "frame_width": _round(fw),
            "iou": _round(iou),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
            "paths": {
                "trajectory": "trajectory.csv",
                "labels": "labels.csv",
                "expected": "expected.json",
            },
            "note": (
                "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
                "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
(§6)."
            ),
        }
```

```
402
403
404     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
405                        iou: float = 0.5) -> Dict[str, Any]:
406         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
407         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
408
409         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
410         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
411         """
412         fdir = fixtures_dir / slug
413         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
414         if not traj_csv.is_file() or not labels_csv.is_file():
415             raise FileNotFoundError(
416                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
417
418         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
419         spec = _spec_by_slug().get(slug)
420         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
421         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
422         _write_json(fdir / "provenance.json", prov)
423
424         expected = replay_fixture(fdir)
425         _write_json(fdir / "expected.json", expected)
426         return expected
427
428
429     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
430         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
431         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
432         specs = builtin_specs()
433         fixtures_dir.mkdir(parents=True, exist_ok=True)
434         slugs: List[str] = []
435         for spec in specs:
436             fdir = fixtures_dir / spec.slug
437             fdir.mkdir(parents=True, exist_ok=True)
438             rows = make_synthetic_trajectory(spec)
439             write_trajectory_csv(rows, fdir / "trajectory.csv")
440             write_labels_csv(spec.gts, fdir / "labels.csv")
441             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
442             slugs.append(spec.slug)
443         _write_manifest(specs, fixtures_dir)
444         return slugs
445
446
447     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
448                          fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
449         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
450         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
451         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
452         if all_slugs:
453             targets = [d.name for d in sorted(fixtures_dir.iterdir())
454                        if d.is_dir() and (d / "trajectory.csv").is_file()]
455         elif slug:
456             targets = [slug]
457         else:
```

```
458             raise ValueError("refresh_snapshot needs --slug S or --all")
459         for s in targets:
460             freeze_fixture(s, fixtures_dir=fixtures_dir)
461         return targets
462 
463 
464     # ============================================================================
465     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
466     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
467     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
468     # ============================================================================
469 
470 
471     class RefusalError(ValueError):
472         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
473 
474         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
it) that
475         names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
476         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
477         / license flags are human-asserted, but the gate makes the unsafe path impossible to
reach
478         *silently* ? a fixture cannot be written unless all three are affirmatively set.
479         """
480 
481 
482     def _new_surrogate_slug() -> str:
483         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
484 
485         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
486         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
487         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
488         """
489         return f"fx_{os.urandom(8).hex()}"
490 
491 
492     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
path tokens
493     #: that collide with these are NOT a leak ? exclude them so a short source stem like
``traj`` can't
494     #: false-positive against the literal
``trajectory.csv``/``labels.csv``/``expected.json``.
495     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
496         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
497         "provenance", "provenance.json",
498     )
499 
500 
501     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
502         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
503 
504         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
505         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
506         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
507         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
```

```
        re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
        """
        low = text.lower()
        for key in FORBIDDEN_KEYS:
            if key in low:
                raise RefusalError(
                    f"{where}: forbidden key substring {key!r} found in written output ? "
                    f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
                    f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
        m = _MD5_RE.search(text)
        if m:
            raise RefusalError(
                f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
                f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
        for sp in source_paths:
            if not sp:
                continue
            p = Path(sp)
            for token in (sp, p.name, p.stem):
                t = (token or "").strip().lower()
                # Skip a token that is itself a substring of a schema-literal the writers
always emit
                # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
                if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
                    continue
                if t in low:
                    raise RefusalError(
                        f"{where}: source-path token {token!r} found in written output ? the
source "
                        f"filename/path must never be persisted (§4c metadata strip).")


    def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
                               ) -> List[Tuple[int, int, float, float]]:
        """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.

        Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
        pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
        """
        rows: List[Tuple[int, int, float, float]] = []
        for p in points:
            rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
        rows.sort(key=lambda r: r[0])
        return rows


    def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
        """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
        return [(s - t0_sec, e - t0_sec) for s, e in gts]


    def freeze_real_fixture(
        label: str,
        trajectory_csv_path: Any,
        labels_csv_path: Any,
        *,
```

```
        license: str,
        minors_free: bool,
        owned: bool,
        fps: float,
        frame_width: float,
        fixtures_dir: Path = FIXTURES_DIR,
        slug: Optional[str] = None,
        time_origin_reset: bool = True,
        source_note: str = "",
    ) -> Dict[str, Any]:
        """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
        Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)

        The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
        ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
        The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
        opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
        reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
        re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
        slipped in.

        NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
        only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
        attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
        owner-asserted Fork-A / minors / biometric-exclusion flags.

        Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
        """
        # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-------
        reasons: List[str] = []
        if minors_free is not True:
            reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
                           "of a minor is prohibited ? §3, guardrail 'exclude minors')")
        if owned is not True:
            reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
                           "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
        if not (isinstance(license, str) and license.strip()):
            reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
                           "'provenance hard-fail gate')")
        if reasons:
            raise RefusalError(
                "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
                "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n  - "
                + "\n  - ".join(reasons))

        traj_path = Path(str(trajectory_csv_path))
        labels_path = Path(str(labels_csv_path))
```

```
609
610         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611         out_slug = slug if slug is not None else _new_surrogate_slug()
612
613         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
614         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615         if not points:
616             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617         gts = load_gt_intervals(str(labels_path), strict=True)
618
619         # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
------
620         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621         # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622         # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623         # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624         if time_origin_reset:
625             t0_frame = min(int(p.frame) for p in points)
626             t0_sec = t0_frame / float(fps)
627         else:
628             t0_frame, t0_sec = 0, 0.0
629         rows = _reset_trajectory_rows(points, t0_frame)
630         reset_gts = _reset_labels(gts, t0_sec)
631
632         # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----------
633         fdir = fixtures_dir / out_slug
634         fdir.mkdir(parents=True, exist_ok=True)
635         write_trajectory_csv(rows, fdir / "trajectory.csv")
636         write_labels_csv(reset_gts, fdir / "labels.csv")
637
638         iou = 0.5
639         prov = {
640             "slug": out_slug,
641             "schema_version": CURRENT_SCHEMA,
642             "kind": "cleared_real",
643             "label": str(label),
644             "license": license,
645             "minors_free": True,
646             "owned": True,
647             "fps": _round(fps),
648             "frame_width": _round(frame_width),
649             "iou": _round(iou),
650             "time_origin_reset": bool(time_origin_reset),
651             "windowing": {
652                 "vt": PINNED_VT,
653                 "mg": PINNED_MERGE_GAP,
654                 "mwd": PINNED_MIN_WINDOW,
655                 "max_win": PINNED_MAX_WINDOW,
656             },
657             "paths": {
658                 "trajectory": "trajectory.csv",
659                 "labels": "labels.csv",
660                 "expected": "expected.json",
661             },
662             "gt_hash": gt_hash(reset_gts),
663             "source_note": str(source_note),
664             "note": (
```

```
665                 "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
666                 "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
667                 "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
668                 "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2)."
669             ),
670         }
671         _write_json(fdir / "provenance.json", prov)
672
673         expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
674         _write_json(fdir / "expected.json", expected)
675
676         # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, §4c / ?)
-----
677         source_paths = [str(traj_path), str(labels_path)]
678         _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
679                         where=f"{out_slug}/expected.json", source_paths=source_paths)
680         _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
681                         where=f"{out_slug}/provenance.json", source_paths=source_paths)
682
683         # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
684         _append_manifest_entry(prov, fixtures_dir)
685         return expected
686
687
688     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
689         """A manifest row for a single fixture, derived from its provenance block."""
690         slug = prov["slug"]
691         return {
692             "slug": slug,
693             "fps": _round(float(prov["fps"])),
694             "frame_width": _round(float(prov["frame_width"])),
695             "windowing": dict(prov.get("windowing", {})),
696             "kind": prov.get("kind", "synthetic"),
697             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
698             "paths": {
699                 "dir": slug,
700                 "trajectory": f"{slug}/trajectory.csv",
701                 "labels": f"{slug}/labels.csv",
702                 "expected": f"{slug}/expected.json",
703                 "provenance": f"{slug}/provenance.json",
704             },
705         }
706
707
708     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
709         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
710         entries = load_manifest(fixtures_dir)
711         entries = [e for e in entries if e.get("slug") != prov["slug"]]
712         entries.append(_manifest_entry_for(prov))
713         entries.sort(key=lambda e: str(e.get("slug")))
714         _write_json(fixtures_dir / "manifest.json", entries)
715
716
717     # ============================================================================
718     # Manifest + loaders
719     # ============================================================================
720
721
```

```
    def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
        entries = [{
            "slug": spec.slug,
            "fps": _round(spec.fps),
            "frame_width": _round(spec.frame_width),
            "windowing": {
                "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
            },
            "kind": "synthetic",
            "schema_version": CURRENT_SCHEMA,
            "paths": {
                "dir": spec.slug,
                "trajectory": f"{spec.slug}/trajectory.csv",
                "labels": f"{spec.slug}/labels.csv",
                "expected": f"{spec.slug}/expected.json",
                "provenance": f"{spec.slug}/provenance.json",
            },
        } for spec in specs]
        _write_json(fixtures_dir / "manifest.json", entries)


    def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
        """Load the fixtures manifest (returns [] if absent)."""
        path = fixtures_dir / "manifest.json"
        if not path.is_file():
            return []
        data = _read_json(path)
        if not isinstance(data, list):  # pragma: no cover - corrupt manifest
            raise ValueError(f"{path}: manifest must be a JSON array")
        return data


    def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
        """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
        whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
        fdir = fixtures_dir / slug
        prov = _read_json(fdir / "provenance.json")
        expected = _read_json(fdir / "expected.json")
        sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
        if sv > CURRENT_SCHEMA:
            raise ValueError(
                f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade
the harness")
        return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}


    # ============================================================================
    # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
    # ============================================================================


    def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
        """Return a list of human-readable mismatch messages (empty == match).

        Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
        TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
        equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
        safe (the repo runs with core.autocrlf=true)."""
        out: List[str] = []
```

```

        def exact(path: str, a: Any, b: Any) -> None:
            if a != b:
                out.append(f"{path}: {a!r} != {b!r} (exact)")

        def approx(path: str, a: Any, b: Any) -> None:
            try:
                if abs(float(a) - float(b)) > _FLOAT_TOL:
                    out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
            except (TypeError, ValueError):
                out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")

        exact("slug", recomputed.get("slug"), committed.get("slug"))
        exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
        exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
        approx("iou", recomputed.get("iou"), committed.get("iou"))

        rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
        if len(rc) != len(cc):
            out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rc, cc)):
                approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
                approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
                rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
                if set(rs) != set(cs):
                    out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
                else:
                    for k in rs:
                        if k == "net_crossings":
                            exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
                        else:
                            approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])

        rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
        if len(rseg) != len(cseg):
            out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rseg, cseg)):
                approx(f"control_segments[{i}][0]", r[0], c[0])
                approx(f"control_segments[{i}][1]", r[1], c[1])

        rsc, csc = recomputed.get("score", {}), committed.get("score", {})
        int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
        float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
                        "mean_start_error", "mean_end_error")
        for k in int_fields:
            exact(f"score.{k}", rsc.get(k), csc.get(k))
        for k in float_fields:
            approx(f"score.{k}", rsc.get(k), csc.get(k))

        return out


    # ============================================================================
    # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
    # ============================================================================


    def _write_json(path: Path, obj: Any) -> None:
        text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
```

```
        path.write_text(text + "\n", encoding="utf-8", newline="\n")


    def _read_json(path: Path) -> Any:
        with open(path, encoding="utf-8") as f:
            return json.load(f)


    def _resolve_dir(slug_or_dir: Any) -> Path:
        if isinstance(slug_or_dir, Path):
            return slug_or_dir
        p = Path(str(slug_or_dir))
        if p.is_dir():
            return p
        return FIXTURES_DIR / str(slug_or_dir)


    def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
        if prov and "fps" in prov and "frame_width" in prov:
            return float(prov["fps"]), float(prov["frame_width"])
        spec = _spec_by_slug().get(slug)
        if spec is not None:
            return spec.fps, spec.frame_width
        raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")


    def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
        prov_path = fdir / "provenance.json"
        prov = _read_json(prov_path) if prov_path.is_file() else None
        return _fps_fw(prov, slug)


    # ============================================================================
    # CLI
    # ============================================================================


    def _cmd_check(fixtures_dir: Path) -> int:
        manifest = load_manifest(fixtures_dir)
        if not manifest:
            print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
            return 1
        failures = 0
        for entry in manifest:
            slug = entry["slug"]
            committed = _read_json(fixtures_dir / slug / "expected.json")
            recomputed = replay_fixture(fixtures_dir / slug)
            mismatches = compare_expected(recomputed, committed)
            if mismatches:
                failures += 1
                print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
                      f"(characterization = behaviour-locking, NOT correctness ? §6):")
                for m in mismatches:
                    print(f"    - {m}")
            else:
                sc = recomputed["score"]
                print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
                      f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
        print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
              f"(synthetic-tier = plumbing correctness, not field quality ? §6)")
        return 0 if failures == 0 else 1
```

```
904
905     def main(argv: Optional[Sequence[str]] = None) -> int:
906         ap = argparse.ArgumentParser(
907             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
908         g = ap.add_mutually_exclusive_group(required=True)
909         g.add_argument("--freeze-synthetic", action="store_true",
910                        help="generate synthetic trajectory+labels for all built-in
scenarios, then "
911                             "freeze expected.json+provenance.json + write manifest.json")
912         g.add_argument("--refresh-snapshot", action="store_true",
913                        help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
914         g.add_argument("--check", action="store_true",
915                        help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
916         g.add_argument("--freeze-real", action="store_true",
917                        help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
918                             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
919         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
920                                        "--freeze-real (default: a random fx_<hex> id)")
921         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
922         ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
diff!)")
923         # --freeze-real (P1 / M2 refusal gate) options:
924         ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
925         ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV
(start,end)")
926         ap.add_argument("--license", dest="license_", default=None,
927                         help="(--freeze-real) the source license tag (required, non-blank)")
928         ap.add_argument("--owned", action="store_true",
929                         help="(--freeze-real) assert owner-recorded Fork-A source
(required)")
930         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
931                         help="(--freeze-real) assert the source contains NO minors
(required)")
932         ap.add_argument("--fps", type=float, help="(--freeze-real) source fps")
933         ap.add_argument("--frame-width", dest="frame_width", type=float, help="(--freeze-
real) source frame width (px)")
934         ap.add_argument("--label", default="cleared_real", help="(--freeze-real) non-
identifying scenario label")
935         ap.add_argument("--source-note", dest="source_note", default="",
936                         help="(--freeze-real) non-identifying note (NEVER a path/name/id)")
937         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
938                         help="(--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
939         ap.set_defaults(time_origin_reset=True)
940         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942         args = ap.parse_args(list(argv) if argv is not None else None)
943
944         fixtures_dir: Path = args.fixtures_dir
945
946         if args.freeze_synthetic:
947             slugs = freeze_synthetic(fixtures_dir)
948             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {',
'.join(slugs)}")
949             print(f"  -> {fixtures_dir}")
```

```
950             return 0
951
952         if args.refresh_snapshot:
953             if not args.slug and not args.all:
954                 ap.error("--refresh-snapshot requires --slug S or --all")
955             if not args.reason:
956                 print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
"
957                       "rubber-stamp a real regression ? review the visible diff (§6).",
file=sys.stderr)
958             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
959             print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
960                   + (f"  (reason: {args.reason})" if args.reason else ""))
961             return 0
962
963         if args.freeze_real:
964             missing = [n for n, v in (("--trajectory", args.trajectory),
965                                       ("--labels", args.labels),
966                                       ("--fps", args.fps),
967                                       ("--frame-width", args.frame_width)) if v is None]
968             if missing:
969                 ap.error("--freeze-real requires " + ", ".join(missing))
970             try:
971                 expected = freeze_real_fixture(
972                     args.label, args.trajectory, args.labels,
973                     license=args.license_ if args.license_ is not None else "",
974                     minors_free=args.minors_free, owned=args.owned,
975                     fps=args.fps, frame_width=args.frame_width,
976                     fixtures_dir=fixtures_dir, slug=args.slug,
977                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
978             except RefusalError as e:
979                 # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
zero.
980                 print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
981                 return 2
982             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
983                   f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
984             print("  §6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
985                   "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
986             return 0
987
988         # --check
989         return _cmd_check(fixtures_dir)
990
991
992     if __name__ == "__main__":
993         raise SystemExit(main())
994
```

5. assistant (2026-06-16T06:33:07.705Z)

This is a detailed numerical claim. Let me read the relevant doc section, the test file, and then empirically verify the claim by running the actual code.

6. user (2026-06-16T06:33:09.650Z)

```
80      `parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates
? rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.
81      Reuses **verbatim**: `metrics.score/interval_iou`, `segmentation_metrics`,
`experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED),
```

`regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0), `eval_baselines/` committed-baseline precedent.

4b. Tiered model (SEALED-FIXTURES)

Tier	Where	Key?	Contents	Test behaviour
Tier-0	`eval_baselines/fixtures/` (tracked)	none	opaque `fixture_id`, fps/frame_width (quantized), relative `start`/`end`, **5 shuttle scalars**, label, relative `gts`. **No person/audio/pose.**	**Unconditional** ? runs on every PR/fork; turns today's skipped litmus into a real public gate
Tier-1	OneDrive (default) **or** SOPS-encrypted in-repo	token/age key	full corpus **with** person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus	**Skips cleanly** when key/data absent (generalize `_golden_present()` ? `_tier1_present()`)

A fresh no-key clone is always **green**; coverage never depends on a secret.

4c. Anti-attribution measures (Tier-0)

- **Opaque RANDOM surrogate id** (`os.urandom` + a private surrogate?source map kept off-git) ? *not* the MD5 `video_id`, and *not* HMAC(salt, video_id?name) (red-team: brute-forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source filename).
- **Metadata strip** ? no filename, video_id, match/event/player identity, capture date.
- **Time-origin reset** ? subtract per-fixture `t0` from all start/end/gts. **Provably metric-invariant** (`metrics.interval_iou` and start/end-error are pure differences) ? bit-identical scores, severed absolute timeline.
- **Stream minimization** ? Tier-0 keeps shuttle block only; person/audio excluded; pose/gait/face have **no field in the schema by design**.
- **Raw-trajectory exclusion** ? never commit the per-frame `(Frame,Visibility,X,Y)` CSV.
- **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue quasi-identifier.
- **Provenance hard-fail gate** ? the de-attribution pass refuses to emit a public fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.

4d. The two gates

- **Characterization/snapshot** ? replay fixture, **parse-compare** JSON (never byte-compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields, `approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints (incommensurable ? loud fail). Catches any behaviour change even at constant F1. **CONTROL path only** across machines.
- **Quality-floor** ? score vs committed labels; assert per-video + mean F1 ? floor in `eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so a served-default change is flagged incommensurable, not silently compared); emit `eval_stats` paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta. Synthetic-tier floors measure **correctness, not field quality** (documented).

4e. Serialization & optional protobuf

JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is optional (O1)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3 `rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`). Protobuf is **encoding, not secrecy** (trivially `--decode_raw`).

4f. Encryption & keys (Tier-1 only)

Default = out-of-repo + token (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS + age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The **HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus, rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.

---

5. Threat model (who's protected vs not)

```
114     | Actor | Protected against | NOT protected (by design / residual) |
115     |---|---|---|
116     | **TM1 no-key public** | map fixture?source/match/player; decrypt Tier-1; confirm a
source even holding the candidate file | read Tier-0 numbers + run shuttle-only harness
(intended; useless-in-isolation) |
117     | **TM2 keyed contributor** | recover filename/video_id/match/face/pose/gait ? *never
written* (survives the analog hole) | read every value the suite reads (unpreventable; not
pretended otherwise) |
118     | **TM3 CI runner** | leaked log can't leak attribution it never had | runs the suite
(intended); Tier-1 secret never on fork runners |
119     | **TM4 malicious keyed insider** | correlation needs a public reference; Fork-A source
is unpublished ? "not reasonably likely" | **residual:** conditional + revocable ? publish/leak
the source later and the link can snap back (forward-looking re-test duty) |
```

7. user (2026-06-16T06:33:09.927Z)

```
1       """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3       These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4       minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5       its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7       Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8       We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9       ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10      prove:
11        * the refusal gate blocks every unsafe combination (M2);
12        * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13        * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14          ``control_segments`` shift toward 0);
15        * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17      §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18      are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19      """
20      import pytest
21
22      from backend.eval import golden_fixtures as gf
23
24
25      # ---------------------------------------------------------------------------------------
26      # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27      # ---------------------------------------------------------------------------------------
28
29
30      def _write_source(tmp_path, *, frame_offset: int = 0):
31          """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33          Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34          ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35          capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36          """
37          spec = gf.builtin_specs()[0]  # clean_single_rally
38          fps, fw = spec.fps, spec.frame_width
```

```
        rows = gf.make_synthetic_trajectory(spec)
        if frame_offset:
            rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
        gts = list(spec.gts)
        if frame_offset:
            off_sec = frame_offset / fps
            gts = [(s + off_sec, e + off_sec) for (s, e) in gts]

        traj = tmp_path / "src_trajectory.csv"
        labels = tmp_path / "src_labels.csv"
        gf.write_trajectory_csv(rows, traj)
        gf.write_labels_csv(gts, labels)
        return traj, labels, fps, fw, gts


    # ---------------------------------------------------------------------------
    # M2 ? refusal gate
    # ---------------------------------------------------------------------------


    @pytest.mark.parametrize(
        "kwargs, why",
        [
            ({"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
            ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
            ({"minors_free": True, "owned": True, "license": ""}, "license=''"),
            ({"minors_free": True, "owned": True, "license": "   "}, "license=blank"),
            ({"minors_free": True, "owned": True, "license": None}, "license=None"),
        ],
    )
    def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
        """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license.

        The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
        must be impossible to reach silently.
        """
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        with pytest.raises(ValueError) as ei:
            gf.freeze_real_fixture(
                "scenario", traj, labels,
                fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
        # RefusalError is a ValueError subclass; the message must name a reason.
        assert isinstance(ei.value, gf.RefusalError), why
        assert "REFUSED" in str(ei.value), why
        # Nothing was written for a refused freeze.
        assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
            f"{why}: a refused freeze must not write any fixture")


    # ---------------------------------------------------------------------------
    # P1 ? de-attribution
    # ---------------------------------------------------------------------------


    def test_real_fixture_is_deattributed(tmp_path):
        """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
        no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        expected = gf.freeze_real_fixture(
```

```
            "clean_rally", traj, labels,
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=fixtures)

        slug = expected["slug"]
        assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"

        prov = gf._read_json(fixtures / slug / "provenance.json")
        assert prov["kind"] == "cleared_real"
        assert prov["minors_free"] is True and prov["owned"] is True
        assert prov["license"] == "owned"
        assert prov["slug"] == slug
        # No attribution keys / no source filename anywhere in the provenance.
        for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
            assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
        prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
        for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
            assert tok not in prov_text, f"source token {tok!r} leaked into provenance"

        # expected.json carries no biometric/identity/non-shuttle stream.
        exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
        for forbidden in gf.FORBIDDEN_KEYS:
            assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"

        # The fixture is registered in the manifest with kind=cleared_real.
        manifest = gf.load_manifest(fixtures)
        rows = [e for e in manifest if e["slug"] == slug]
        assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"


    def test_supplied_slug_is_honored(tmp_path):
        """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        expected = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_custom00",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=fixtures)
        assert expected["slug"] == "fx_custom00"
        assert (fixtures / "fx_custom00" / "expected.json").is_file()


    def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
        """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
                                    license="owned", minors_free=True, owned=True,
                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
        e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
                                    license="owned", minors_free=True, owned=True,
                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
        slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
        assert {e1["slug"], e2["slug"]} <= slugs


    def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
        """source_note is non-identifying by contract ? a path/name in it trips the positive
scan."""
```

```
        fixtures = tmp_path / "fixtures"
        traj, labels, fps, fw, _ = _write_source(tmp_path)
        with pytest.raises(gf.RefusalError):
            gf.freeze_real_fixture(
                "scenario", traj, labels, slug="fx_leak0001",
                license="owned", minors_free=True, owned=True,
                fps=fps, frame_width=fw, fixtures_dir=fixtures,
                source_note=str(traj))  # leaking the source path ? must be refused


    # ---------------------------------------------------------------------------------------
    # P1 (d) ? time-origin reset is metric-invariant
    # ---------------------------------------------------------------------------------------


    def test_time_origin_reset_is_metric_invariant(tmp_path):
        """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
frozen
        WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
timeline)."""
        offset_frames = 9000  # +5 minutes at 30fps ? a large absolute capture-timeline
offset
        traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)

        with_reset = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_reset_on",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
            time_origin_reset=True)
        without_reset = gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_reset_off",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
            time_origin_reset=False)

        # (1) Score is bit-identical ? metrics.score is built from pure differences.
        assert with_reset["score"] == without_reset["score"], (
            "time-origin reset must be metric-invariant (score built from pure
differences)")

        # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
        seg_on = with_reset["control_segments"]
        seg_off = without_reset["control_segments"]
        assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
        assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
        # The shift equals the offset in seconds (within rounding).
        shift = seg_off[0][0] - seg_on[0][0]
        assert abs(shift - offset_frames / fps) < 1e-3, (
            f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
        # Reset trajectory starts at frame 0.
        first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
                          .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
        assert first_frame == 0, "with reset, the trajectory must start at Frame 0"


    # ---------------------------------------------------------------------------------------
    # P1 (e/f) ? round-trip / correct-by-construction
    # ---------------------------------------------------------------------------------------


    def test_freeze_real_round_trips_check(tmp_path):
        """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
```

```
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",
240                 license="owned", minors_free=True, owned=True,
241                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
242
```

8. assistant (2026-06-16T06:33:17.578Z)

Now I understand the structure. The claim is that the time-origin reset is NOT metric-invariant for non-whole-second t0, because t0_sec is computed as `t0_frame / fps` in float, then labels are written at 6dp, and the strict loader re-parses the rounded text. Let me verify this empirically by running the actual code with a sweep of frame offsets.

Let me first check the metrics computation and gt_loader to understand the full path.

9. user (2026-06-16T06:33:19.645Z)

Python 3.13.13

10. assistant (2026-06-16T06:33:22.360Z)

numpy is available. Let me check whether the full CONTROL replay path actually avoids numpy (the doc says it does) so I can run the empirical sweep here.

11. user (2026-06-16T06:33:26.806Z)

import OK

12. assistant (2026-06-16T06:33:29.934Z)

Now let me run the empirical sweep that the finding describes: freeze the same source with `time_origin_reset=True` vs `False` across frame offsets 0..39 at fps=30, and check score equality.

13. user (2026-06-16T06:33:42.895Z)

offsets where score differs: [1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20, 22, 23, 25, 26, 28, 29, 31, 32, 34, 35, 37, 38]

```
count broken: 26 / 40
worst delta: 1.000000000001e-06 at (1, 'mean_start_error', 0.033333, 0.033334)
_FLOAT_TOL: 1e-06 | worst > TOL: True
```

14. assistant (2026-06-16T06:33:49.823Z)

The empirical sweep reproduces the finding exactly: 26/40 offsets break score equality, and the worst delta is `1.000000000001e-06`, strictly greater than `_FLOAT_TOL=1e-6`. Now let me verify the critical second half of the claim: that this worst delta actually flips the `compare_expected.approx` result (i.e., that it would cause a `--check` FAIL, not just a difference between two `freeze` outputs).

The real question for `isMustFix` is: can this make the committed `--check` gate flip across machines? Let me test whether `compare_expected` actually flags the offset=1 case, and crucially, whether the worst delta exceeds the tolerance in `approx`.

15. user (2026-06-16T06:33:57.363Z)

```
delta = 1.000000000001e-06
TOL   = 1e-06
approx flags mismatch (d > TOL)? True
compare_expected approx() result: ['score.mean_start_error: 0.033333 != 0.033334']
```

16. assistant (2026-06-16T06:34:03.294Z)

Confirmed: the worst-case delta `1.000000000001e-06` is strictly greater than `_FLOAT_TOL`, so `compare_expected.approx()` flags it as a mismatch. Now, the key question for whether this is a *real* must-fix in the shipped suite: the `--check` gate compares a *recomputed* replay against a *committed* `expected.json`. Both come from the same committed `trajectory.csv`/`labels.csv` via `replay_fixture`, which does NOT apply the reset (the reset happens only at freeze time). So `--check` itself is deterministic given committed bytes.

Let me think carefully about where the non-invariance actually bites. The claim is about `time_origin_reset=True vs False` producing different scores ? i.e., the de-attribution transform is not score-preserving. Let me verify the actual mechanism: is it the label rounding, or the trajectory frame shift, or the `score` computation? Let me trace exactly what differs.

17. user (2026-06-16T06:34:13.759Z)

```
--- source labels.csv ---
start,end
0.233333,4.233333

t0_sec = 0.23333333333333334 -> _fmt: 0.233333
--- reset labels.csv ---
start,end
0.000000,4.000000

--- no-reset labels.csv ---
start,end
0.233333,4.233333

score WITH reset   mean_start_error: 0.033333
score WITHOUT reset mean_start_error: 0.033334
```

18. assistant (2026-06-16T06:34:22.857Z)

Now I fully understand the mechanism. The root cause is confirmed:

- Source label start = `0.233333` (the source CSV is itself rounded to 6dp at write time).
- WITH reset: `t0_sec = 7/30 = 0.2333...`, label becomes `0.233333 - 0.2333... = 0.0000...` ? written as `0.000000`. The reset label start error vs the candidate window start (0.033333...) gives `0.033333`.
- WITHOUT reset: label stays `0.233333`; the error vs the candidate window gives `0.033334`.

The asymmetry arises because in the reset path the full-precision `t0_sec` is subtracted (cancelling the rounding cleanly to `0.0`), but in the no-reset path the *already-6dp-rounded* source label (`0.233333`, which differs from the true `0.2333...` by ~3.3e-7) flows through, and that residual rounding shifts the start-error across the 6dp boundary. The two paths genuinely disagree by `1e-6 + epsilon`.

So both halves of the finding are confirmed:
1. The "provably metric-invariant / bit-identical scores" doc/code claim is contradicted for non-whole-second t0.
2. The worst measured delta strictly exceeds `_FLOAT_TOL`.

Now the critical `isMustFix` question. Let me assess the *actual operational impact* ? does this break the shipped suite or a real gate? Let me check what the committed test actually does and whether any real (non-synthetic) `freeze_real` fixture path is exercised in CI.

19. user (2026-06-16T06:34:28.151Z)

```
............                                                             [100%]
12 passed in 2.49s
```

20. assistant (2026-06-16T06:34:31.097Z)

The committed test passes. Now let me verify the finding's claim that the test only passes because of the hard-coded 9000-frame (whole-second) offset. Let me check: does the test suite anywhere exercise a fractional offset? And what does `--check` actually compare (recomputed vs committed, both from same bytes)?

21. user (2026-06-16T06:34:32.444Z)

```
tests\test_golden_real_fixtures.py:30:def _write_source(tmp_path, *, frame_offset: int = 0):
tests\test_golden_real_fixtures.py:34:    ``frame_offset`` frames (and the matching label
seconds) to model an arbitrary absolute
tests\test_golden_real_fixtures.py:40:    if frame_offset:
tests\test_golden_real_fixtures.py:41:        rows = [(f + frame_offset, v, x, y) for (f, v, x,
y) in rows]
tests\test_golden_real_fixtures.py:43:    if frame_offset:
tests\test_golden_real_fixtures.py:44:        off_sec = frame_offset / fps
tests\test_golden_real_fixtures.py:172:def test_time_origin_reset_is_metric_invariant(tmp_path):
tests\test_golden_real_fixtures.py:175:    offset_frames = 9000  # +5 minutes at 30fps ? a large
absolute capture-timeline offset
tests\test_golden_real_fixtures.py:176:    traj, labels, fps, fw, _ = _write_source(tmp_path,
frame_offset=offset_frames)
tests\test_golden_real_fixtures.py:182:        time_origin_reset=True)
tests\test_golden_real_fixtures.py:187:        time_origin_reset=False)
tests\test_golden_real_fixtures.py:200:    assert abs(shift - offset_frames / fps) < 1e-3, (
tests\test_golden_real_fixtures.py:201:        f"segment shift {shift} should equal the frame
offset in seconds {offset_frames / fps}")
```

22. assistant (2026-06-16T06:34:37.021Z)

Confirmed: the only invariance test uses `offset_frames = 9000` (= 300.0s exactly, a whole-second offset). 9000/30 = 300.0, so `t0_sec` is exact and the reset cleanly cancels. Let me prove the test would fail at a fractional offset, to validate the finding's "only passes because of the whole-second offset" claim.

23. user (2026-06-16T06:34:48.052Z)

```
offset=    7  t0_sec=0.23333333  score-equal(committed-test-assert)=False
offset=   11  t0_sec=0.36666667  score-equal(committed-test-assert)=False
offset=   13  t0_sec=0.43333333  score-equal(committed-test-assert)=False
offset= 9000  t0_sec=300.00000000  score-equal(committed-test-assert)=True
```

24. assistant (2026-06-16T06:34:59.265Z)

Fully confirmed. The committed test's assertion `with_reset["score"] == without_reset["score"]` (test_golden_real_fixtures.py:190) holds only for the whole-second 9000-frame offset and would fail for fractional offsets 7, 11, 13.

Now let me weigh `isMustFix` honestly. The finding's severity claim hinges on whether this can flip a *shipped gate*. Let me check the operational reality:

1. The doc claim at line 94 ("Provably metric-invariant ? bit-identical scores") and the code docstring at golden_fixtures.py:619-623 + 583-586 are **factually wrong** for non-whole-second t0. That is a real, demonstrable falsehood.

2. The `--check` gate (`_cmd_check`) compares a *recomputed* replay against the *committed* `expected.json` ? both derived from the **same committed bytes** via `replay_fixture`, which never applies the reset. So once a fixture is frozen, `--check` is internally deterministic; the non-invariance does NOT flip `--check` across machines for an already-committed fixture.

3. Where it DOES bite: the *claim* that the reset (de-attribution) preserves the score. A real `freeze_real` fixture committed with reset will carry a score that differs (by up to ~1e-6+?) from what the un-reset source would have produced. Since the score is baked into `expected.json` at freeze time and `--check` recomputes from the same reset bytes, the committed fixture is self-consistent ? the regression gate still works. The harm is narrower: the *anti-attribution transform is not score-neutral*, contradicting a documented invariant, and there is no test guarding the fractional-offset case.

Let me confirm point 2 ? that `--check` recomputes from committed (already-reset) bytes and is therefore self-consistent regardless of the reset path.

25. user (2026-06-16T06:35:08.250Z)

--check (recompute vs committed, same reset bytes) mismatches: []
=> committed fixture is self-consistent under --check: True

26. assistant (2026-06-16T06:35:26.030Z)

This confirms the nuance: the shipped `--check` gate (the cross-machine snapshot gate) is self-consistent and does NOT break, because it recomputes from the same committed (already-reset) bytes. The non-invariance is between the reset and no-reset *freeze* outputs ? i.e., the de-attribution transform is not score-neutral, contradicting the documented "bit-identical scores" claim.

I now have everything needed for a verdict. Let me summarize my assessment.

Both factual halves of the finding are CONFIRMED:
- The "Provably metric-invariant ? bit-identical scores" claim (docs/GOLDEN_REGRESSION_FIXTURES.md:94, and the code docstrings at golden_fixtures.py:619-623 + 583-586, and the test docstring at test_golden_real_fixtures.py:13-14, 189-191) is **false** for non-whole-second t0: 26/40 frame offsets break score equality.
- The worst measured delta is `1.000000000001e-06`, which is **strictly greater than `_FLOAT_TOL=1e-6`** (golden_fixtures.py:91), and `compare_expected.approx` (golden_fixtures.py:786-789) does flag it (`>` not `>=`).
- The committed test passes only because `offset_frames=9000` ? `300.0s` exactly (test_golden_real_fixtures.py:175); fractional offsets (7, 11, 13) make the test's own assertion fail.

On `isMustFix` ? this is where I push back on the "high severity / can flip a result" framing:
- The shipped cross-machine snapshot gate (`--check` / `_cmd_check`) recomputes from the **same committed reset bytes** and is self-consistent ? I verified it returns `[]` even at the worst-case offset=7. So this bug does **not** flip the `--check` regression gate for any already-committed fixture, and does not make the suite flaky across machines.
- There are **no committed real (`cleared_real`) fixtures** that would exercise the reset path with a fractional offset; the synthetic fixtures never use the reset, and `freeze_real` is owner-gated P1 tooling behind counsel sign-off (not run in CI). So no green/red gate currently flips.
- The real defect is: (a) a **documented invariant that is provably false** (a

correctness/credibility issue in a doc that explicitly says "Provably"), and (b) a **test that asserts the invariant but only on the one offset that hides the bug** ? a genuine test-coverage gap. The proposed fix (round t0_sec to 6dp before subtracting from both streams, or reset by integer frames consistently, plus a fractional-offset parametrized test) is correct and appropriate.

This is a real, reproducible issue worth fixing before anyone relies on the "metric-invariant de-attribution" guarantee ? but it is **not** currently breaking the test suite or flipping a CI gate (the suite is green and self-consistent). It is a false documented-invariant + a misleading test, not an active suite-breakage. I'll mark it confirmed but isMustFix=false, since nothing relied-upon is currently broken and the affected `freeze_real` path is owner-gated and unused in CI ? though it should be fixed before the reset guarantee is depended on.

27. user (2026-06-16T06:35:41.402Z)

Structured output provided successfully

28. assistant (2026-06-16T06:35:51.925Z)

I've completed the verification and submitted my structured verdict.

Summary: The finding is **confirmed** on every factual point, verified by running the actual worktree code:

- **Non-invariance reproduced:** 26/40 frame offsets break reset-vs-no-reset score equality (exactly the offsets listed). Root cause traced: source `labels.csv` is written at 6dp, and the no-reset path carries that ~3.3e-7 rounding residual through the start-error while the reset path cancels it cleanly via full-precision `t0_sec`.
- **Past the cliff:** worst `|delta| = 1.0000000000001e-06`, strictly greater than `_FLOAT_TOL=1e-6`, and `compare_expected.approx` (`>` comparison) does flag it.
- **Test masks it:** the lone invariance test hard-codes `offset_frames=9000` (=300.0s exactly). I reran its assertion at offsets 7/11/13 ? all fail; only 9000 passes.

But isMustFix=false, because the finding's "high / can flip a result" framing overstates the impact. The shipped `--check` gate recomputes from the same committed (already-reset) bytes, so it is self-consistent ? I confirmed it returns clean `[]` even at the worst-case offset=7. There are no committed `cleared_real` fixtures exercising the reset path, and `freeze_real` is owner-gated P1 tooling not run in CI. So nothing currently relied upon is broken or flaky; the real defects are a provably-false documented invariant and a coverage gap, worth fixing before the metric-invariant de-attribution guarantee is depended on ? not an active suite breakage.

Relevant files:
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\backend\eval\golden_fixtures.py` (lines 91, 274-279, 552-554, 619-630, 786-789)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\docs\GOLDEN_REGRESSION_FIXTURES.md` (line 94)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\tests\test_golden_real_fixtures.py` (lines 172-205, esp. 175 and 190)